

# COMPREHENSIVE ASSESSMENT REPORT

## (DRAFT)

---

**Module:** Computational Intelligence (CIS 6005)

**Assessment:** WRIT1 - Deep Learning Plus AI Mini Project

**Target Dataset:** Real Estate House Price Prediction (`estate_train.csv`)

**Trained Models:** Linear Regression, XGBoost Regressor, Random Forest Regressor

**Artifact Link:** [submission.csv](#)

**Application Code:** [app.py](#) | [templates/index.html](#)

---

## Section A: Computational Intelligence vs Traditional Artificial Intelligence (10 Marks)

---

### 1. Introduction to Artificial Intelligence Paradigms

Artificial Intelligence (AI) as a scientific discipline seeks to create computational systems that exhibit cognitive behaviors analogous to human intelligence (Russell and Norvig 2020). Historically, the pursuit of AI has bifurcated into two primary paradigms: **Traditional Artificial Intelligence** (also known as Symbolic AI, Hard AI, or Good Old-Fashioned AI - GOFAI) and **Computational Intelligence** (CI, often associated with Soft Computing or Connectionist AI). While Traditional AI focuses on high-level cognitive functions through symbolic manipulation, Computational Intelligence approaches problem-solving from a low-level, biologically-inspired, and data-driven perspective (Engelbrecht 2007).

---

### 2. Traditional Artificial Intelligence (Symbolic & Expert Systems)

Traditional AI is characterized by a "top-down" conceptual framework. It operates on the physical symbol system hypothesis, which posits that symbolic processing is necessary and sufficient for general intelligent action (Newell and Simon 1976).

- **Core Tenets and Mechanisms:** Traditional AI models human reasoning by explicitly coding rules and knowledge. The core mechanism involves a **Knowledge Base** (a repository of facts and rule-based logic, typically represented as IF-THEN structures) and an **Inference**

**Engine** (which applies logical rules using deductive reasoning mechanisms such as forward-chaining or backward-chaining).

- **Strengths:** Because reasoning is representational and rule-based, symbolic systems are highly deterministic and explainable. The logic path leading to a specific decision can be fully traced and audited, making it highly effective in closed, deterministic, and highly structured environments (e.g., mathematical theorem proving, tax calculators, and chess engines).
  - **Critical Limitations:** Symbolic systems exhibit extreme brittleness (McCarthy 1990). They require absolute certainty and structured inputs; they lack the capacity to process noisy, ambiguous, or incomplete real-world data. Furthermore, they suffer from the "**Knowledge Acquisition Bottleneck**"—the practical impossibility of manually coding every potential rule in complex, dynamically changing environments.
- 

### 3. Computational Intelligence (CI)

In contrast, Computational Intelligence (CI) utilizes a "bottom-up" approach, focusing on adaptive mechanisms that learn from raw data or interactive environments. Bezdek (1994) famously distinguished CI from traditional AI by stating that a system is computationally intelligent when it begins to process low-level numeric pattern data, incorporates learning and adaptation, and avoids rigid symbolic representation.

The primary pillars of Computational Intelligence include: 1. **Artificial Neural Networks (ANNs):** Connectionist models inspired by biological nervous systems. They consist of layers of interconnected nodes (neurons) that adjust their synaptic weights via training algorithms (e.g., backpropagation) to approximate complex, highly non-linear functions (Rumelhart, Hinton and Williams 1986). 2. **Evolutionary Computation (EC):** Optimization and search paradigms inspired by biological evolution and genetics (e.g., Genetic Algorithms). They generate optimal solutions through iterative generation, selection, crossover, and mutation (Holland 1992). 3. **Fuzzy Logic Systems (FLS):** A mathematical framework designed to process "approximate" rather than "exact" reasoning. Unlike binary Boolean logic, fuzzy logic maps membership inputs to continuous truth values between 0 and 1, simulating human decision-making under uncertainty (Zadeh 1965).

---

### 4. Comparative Evaluation: Traditional AI vs. Computational Intelligence

The fundamental differences between the two paradigms can be evaluated across several critical operational dimensions:

| Dimension              | Traditional Artificial Intelligence (Symbolic AI)                                      | Computational Intelligence (Soft Computing)                                                         |
|------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Logic & Precision      | Binary Boolean logic (True/False; 0 or 1). Demands high precision.                     | Multi-valued fuzzy logic. Tolerates imprecision, noise, and approximation.                          |
| Reasoning Flow         | <b>Top-Down:</b> Deductive reasoning from general rules to specific cases (Deduction). | <b>Bottom-Up:</b> Inductive learning from specific data points to general patterns (Induction).     |
| Knowledge Origin       | Manually engineered and hard-coded by human domain experts.                            | Learned dynamically and autonomously from empirical data.                                           |
| Adaptability           | Static. System rules must be manually recoded to accommodate changes.                  | Dynamic. Adapts self-correctively through weight adjustments or evolution.                          |
| Explainability         | <b>White-Box:</b> Decisions are highly transparent and logical.                        | <b>Black-Box:</b> High mathematical complexity makes feature interactions difficult to audit.       |
| Real-world Suitability | Structured, rule-bound systems (e.g., database management, syntax parsing).            | Uncertain, non-linear, spatial, and noisy tasks (e.g., house price forecasting, image recognition). |

### Suitability for Housing Price Forecasting

For the task of real estate valuation, traditional symbolic systems are highly deficient. Real estate prices are governed by highly non-linear correlations, geographical coordinate interactions, and macro-economic volatility that cannot be expressed as manual IF-THEN rules. By utilizing a Computational Intelligence approach (such as **XGBoost** or **Random Forest** ensembles), the system can autonomously ingest spatial datasets (`estate_train.csv`), model the complex non-linear interaction of attributes (like mapping Latitude/Longitude coordinates onto prices), and

generalize effectively to predict prices on unseen test cases (`estate_test.csv`).

---

## 5. References

- Bezdek, J.C., 1994. What is computational intelligence? *Computational Intelligence: Imitating Life*, pp.1-12.
  - Engelbrecht, A.P., 2007. *Computational intelligence: an introduction*. John Wiley & Sons.
  - Holland, J.H., 1992. *Adaptation in natural and artificial systems*. MIT press.
  - McCarthy, J., 1990. Formalizing common sense. *Formalizing Common Sense: Papers by John McCarthy*, pp.93-116.
  - Newell, A. and Simon, H.A., 1976. Computer science as empirical inquiry: Symbols and search. *Communications of the ACM*, 19(3), pp.113-126.
  - Rumelhart, D.E., Hinton, G.E. and Williams, R.J., 1986. Learning representations by back-propagating errors. *Nature*, 323(6088), pp.533-536.
  - Russell, S. and Norvig, P., 2020. *Artificial intelligence: a modern approach*. 4th ed. Pearson.
  - Zadeh, L.A., 1965. Fuzzy sets. *Information and Control*, 8(3), pp.338-353.
- 

## Section B: Literature Review (20 Marks)

---

The use of machine learning and deep learning in real estate decision-making, especially in property valuation and housing price prediction, has received a lot of attention in recent years. Traditional mass appraisal relied on statistical models and expert-defined rules (such as simple price-per-square-foot metrics), which often lacked flexibility and struggled to recognize complex relationships among geographic and structural attributes. As large real estate datasets and powerful computing resources became available, machine learning and deep learning methods have increasingly taken the place of conventional methods, providing better predictive accuracy and reliability.

Early research in house price prediction used classical machine learning algorithms like linear regression and decision trees. Linear regression has been popular in hedonic property pricing because it is easy to interpret and simple to use. However, some studies have noted its limitations in managing non-linear feature interactions and high-dimensional spatial data (Lessmann, 2015).

Decision tree models improved this by capturing non-linear relationships but were prone to overfitting when used alone.

Ensemble learning methods, especially Random Forests and Gradient Boosting Machines, represented a significant leap in real estate valuation modeling. Random Forest regressors combine predictions from multiple decision trees, which helps reduce variance and improve overall performance. Breiman's research (2001) showed that these ensemble methods consistently outperform single regressors in structured tabular datasets (Breiman, 2001). Later studies in property price prediction confirmed that Random Forest models perform well when handling mixed numerical and categorical features, as well as spatial distributions often seen in housing datasets (Tianqi Chen, 2016).

Support Vector Machines (SVMs) have also been investigated for property valuation and price prediction. Their strength lies in their ability to map features to high-dimensional spaces using kernel functions to create optimal regression fits. However, SVMs require careful selection of kernels and tuning of hyperparameters, and their scalability is often limited when working with large datasets. As a result, while SVMs show good performance in controlled experimental settings, they are less frequently used in large-scale production compared to ensemble or neural-based models.

The rise of deep learning shifted predictive modeling by allowing systems to automatically learn hierarchical feature representations. Artificial Neural Networks (ANNs) are widely used in financial forecasting and real estate valuation because they can model complex non-linear relationships. Goodfellow, Bengio, and Courville (2016) pointed out that deep neural networks reduce the need for extensive manual feature engineering, which is particularly useful in areas with diverse data types. In real estate pricing, neural networks have demonstrated stronger predictive power when enough data is available, especially in understanding subtle interactions among structural, demographic, and geographical variables (Ian Goodfellow, 2016).

LeCun, Bengio, and Hinton (2015) noted that deep learning excels when patterns are highly non-linear and spread across multiple features—conditions that fit well with housing and coordinate datasets. Recent studies have applied multilayer perceptrons (MLPs) to house valuation tasks, showing competitive or even better performance compared to traditional machine learning models, especially when paired with effective preprocessing and regularization. However, deep learning models often lack interpretability, which is a significant issue in regulated real estate valuation areas where explanations for property tax or loan approval are required (Yann LeCun, 2015).

Several comparative studies have looked at different machine learning architectures within one framework to find the best approaches for housing price prediction. For instance, Lessmann et al.

(2015) conducted a large benchmark analysis and concluded that ensemble and neural-based models outperform linear classifiers and regressors across various datasets. These results support using hybrid evaluation strategies, where classical models serve as baselines while more complex models are tested for performance improvements (Lessmann, 2015).

In Kaggle housing prediction competitions, similar methods can be seen. Participants often use a mix of linear regression, Random Forests, gradient boosting, and neural networks to achieve a good balance between interpretability, accuracy, and generalization. The "Real Estate Price Prediction" competition follows this trend, encouraging experimentation with both classical and deep learning methods. The evaluation metric, R-squared ( $R^2$ ), stresses the importance of variance-based prediction accuracy over binary outputs, which aligns with best practices in real estate appraisal modeling.

In summary, the literature clearly shows that while traditional machine learning techniques are valuable for baseline modeling and interpretability, ensemble and deep learning methods provide better performance in complex real-world real estate datasets. Combining neural networks or gradient boosting with classical models creates a solid evaluation framework, allowing informed model selection based on actual performance rather than theoretical assumptions alone. This project builds on these research findings by implementing, comparing, and deploying multiple learning techniques in a real-world Kaggle competition scenario.

---

## References

- Breiman, L., 2001. Random forests. *Machine Learning*, 45(1), pp.5-32.
  - Goodfellow, I., Bengio, Y. and Courville, A., 2016. *Deep learning*. MIT press.
  - LeCun, Y., Bengio, Y. and Hinton, G., 2015. Deep learning. *Nature*, 521(7553), pp.436-444.
  - Lessmann, S., Baesens, B., Seow, H.V. and Thomas, L.C., 2015. Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. *European Journal of Operational Research*, 247(1), pp.124-136.
  - Limsombunchao, V., 2004. House price prediction: hedonic price model vs. artificial neural network. *New Zealand Agricultural and Resource Economics Society Conference*, pp.25-26.
  - Tianqi Chen, C.G., 2016. Xgboost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp.785-794.
-

## Section C: Exploratory Data Analysis (EDA) and Model Influence (10 Marks)

---

### 1. Missing Value and Outlier Analysis

- **Missing Values:** We dropped rows containing missing `PropertyAge` (1,313 rows in the training set) to preserve quality. Features like `TotalBedrooms` had median imputation applied.
- **Outliers:** IQR and 3-sigma calculations identified skewness in `AvgOccupancy` and `RoomsPerHousehold`. This influenced the decision to use a Robust Pipeline containing `SimpleImputer(strategy='median')` and `StandardScaler` to normalize distributions before feeding them to models.

### 2. Feature Binning and Statistical Justification

- `PropertyAge` was binned into `PropertyAge_bins` ('New' <= 15, 'Moderate' <= 35, 'Old' > 35) to handle non-linear real estate depreciation.
- **ANOVA Test:** We ran an Analysis of Variance (ANOVA) test (`f_oneway`) to verify if the binned age groups have statistically different mean target prices. The test resulted in a  $p\text{-value} = 0.0000$  ( $F\text{-statistic} \gg 1$ ), confirming that binning `PropertyAge` was highly significant and statistically valid.
- **T-Test:** A T-test (`ttest_ind`) between 'New' and 'Old' properties confirmed that new houses have a significantly different price profile from old ones ( $p = 0.0000$ ).

### 3. Correlation Matrix Insights

- `IncomeLevel` showed a strong positive correlation ( $r \approx 0.69$ ) with `TargetPrice`, making it the single most predictive feature.
  - `Latitude` and `Longitude` showed complex spatial distributions, indicating that location coordinates interact with Demographic parameters, calling for ensemble models that handle multi-dimensional thresholds.
-

## Section D: System Architecture (10 Marks)

### 1. Processing Pipeline

```
graph TD
    A[estate_train.csv / Raw Data] --> B[Data Cleaning: Drop ID, Drop PropertyAge NaNs]
    B --> C[Feature Engineering: PropertyAge Binning]
    C --> D[ColumnTransformer Preprocessing: Imputation + StandardScaler + OrdinalEncoder]
    D --> E[Train-Test Split 80/20]
    E --> F[GridSearchCV Hyperparameter Tuning]
    F --> G[Linear, RF, and XGBoost Best Estimators]
    G --> H[Model Serialization: joblib.dump]
    H --> I[Flask Web Server: app.py]
    J[estate_test.csv] --> K[Impute Test Age via Train Median]
    K --> L[Test set Binning]
    L --> M[Fitted Preprocessor Transform]
    M --> N[Predict TargetPrice using Saved XGBoost Model]
    N --> O[submission.csv]
```

### 2. Model Selection Justification

- Linear Regression:** Baseline regression model, fast and simple but restricted to linear relationships.
- Random Forest:** Averages many independent decision trees. By introducing randomness during tree splitting, it reduces variance, increases stability, and provides robust predictions.
- XGBoost:** A powerful gradient boosted decision trees algorithm. It trains trees sequentially, where each tree corrects errors of the previous ones. It is highly optimized, handles complex non-linear relationships, and achieves outstanding accuracy.

## Section E: Full Model Evaluation & Implementation (40 Marks)

### 1. Performance Results (Test Set)

The models were trained using 6-fold cross-validation on the training set and evaluated on the test set:

| Model | Tuning Hyperparameters | $R^2$ Score | Root Mean Squared Error (RMSE) | Mean Absolute Error (MAE) |
|-------|------------------------|-------------|--------------------------------|---------------------------|
|-------|------------------------|-------------|--------------------------------|---------------------------|



|                          |                                                             |               |               |               |
|--------------------------|-------------------------------------------------------------|---------------|---------------|---------------|
| <b>Linear Regression</b> | None                                                        | 0.6521        | 0.6693        | 0.4805        |
| <b>Random Forest</b>     | n_estimators: 200,<br>max_depth: 12,<br>min_samples_leaf: 2 | 0.7854        | 0.5257        | 0.3522        |
| <b>XGBoost</b>           | n_estimators: 200,<br>max_depth: 6,<br>learning_rate: 0.1   | <b>0.8256</b> | <b>0.4739</b> | <b>0.3124</b> |

## 2. Web Application Setup

- **Backend (app.py):** Utilizes Flask. Loads serialized `preprocessor.joblib`, `linear_regression_model.joblib`, `xgboost_model.joblib`, and `random_forest_model.joblib` on startup. Serves `/predict` endpoint returning JSON predictions.

**Frontend (templates/index.html):** Glassmorphic web panel with Outfit typography and dynamic dark mode sliders for all 10 features:

- *Direct sliders:* IncomeLevel, PropertyAge, TotalRooms, TotalBedrooms, AvgOccupancy, RoomsPerHousehold, BedroomsRatio.
- *Direct inputs:* NeighborhoodPop, Latitude, Longitude.
- Live updates are sent asynchronously to the backend on slider adjust, updating predicted prices dynamically with a neon green layout.

---

## Section F: Critical Evaluation & Limitations (10 Marks)

---

### 1. Domain Suitability of the Model

- The Random Forest Regressor is highly suited for real estate valuation. It is robust to outliers (which are common in real estate due to luxury housing spikes) and captures the complex interactions between geographic coordinates and average occupancy.

## 2. Suitability of Deep Learning

- While Deep Learning (e.g. MLPs) can approximate the spatial price function, it is less suitable here due to:
  - *Data scale*: Tabular dataset size (15k rows) is too small to train large network parameters without massive overfitting.
  - *Interpretability*: Real estate appraisers require structural rules (e.g., how the bedrooms ratio impacts price) which are inspectable in tree splits but completely hidden in deep neural weights.

## 3. Limitations of the Current Approach

- **Temporal Shift**: The model does not include time metrics. Inflation, interest rates, and seasonal spikes are completely ignored.
- **Geographical Constraints**: The model is bound to the coordinate boundaries of the training dataset. It cannot predict prices for properties outside these geographical boundaries.

## 4. Future Suggestions

- We successfully implemented XGBoost and improved the  $R^2$  score to **0.8256**. Future work could incorporate LightGBM to speed up training.
- Incorporate SHAP (Shapley Additive exPlanations) values to provide local feature contribution visualization in the web application UI.